MLB Punkte für A: 0:3 → 0
1:2 → 1
2:1 → 3
3:0 → 4

Elo System:

- Erwartungsfunktion $E(R_A, R_B)$: gegeben Ratings R_A, R_B, was sind erwarte Punkte für A
- Update Funktion $U(P_{A,B})$ gegen Ausgang von Partie zwische A,B (P_A, P_B), was ist Update von Ratings

$$E(R_A, R_B) = \frac{4}{1+10^{(R_B-R_A)/R_C}}$$

, R_C: Kritischer Wert, ab dem Sieg sehr wahrscheinlich (d.h. 4 Punkte [oder nur 3?])

$$U(P_{A,B}) = \begin{bmatrix} k(S_A - E(R_A,R_B)) \\ k(S_B - E(R_B,R_A)) \end{bmatrix}$$

S: Erspielte Punkte für A/B, k: Volatilitätsfaktor. Im Schach $k \in [10, 40]$ Profis

k wandelt Erwartungswert $\in (0,1)$ in Ranking um.
Schach: 900 ich, 2800 Magnus Carlsen
Plan: Balatro: 2$ ich, 8$ Dr Spectred $\Rightarrow k_{balatro} = \underset{[10,40]}{k_{schach}} \cdot \frac{8}{2800}$

$$\begin{bmatrix} R_A' \\ R_B' \end{bmatrix} = \begin{bmatrix} R_A \\ R_B \end{bmatrix} + U(P_{A,B})$$

Umsetzung

Objekt
```
Player
rating :: Float
name :: String
cost :: Float
```
→ mutable

Objekt
```
Match
player1 :: String
player2 :: String
```

Objekt
```
Result
match :: Match
player1Score :: Int
```

Funktion
```
expectedScore(match :: Match; player :: Int = 1) = E
rankingUpdate(result :: Result) = U
updateRank!(result :: Result; which :: Union{Int, Vector{Int}} = 1)
    → R' = R + U
```

Brauche Funktion, die alle Matches (aus ner Excel?) einliest und
Vector matchday :: Vector{Match} S.U.
ausgibt. Gleiches für result → results :: Vector{Result}

Dann:
```
for result in results
    updateRank!(result)
end
```

I/O

CSV mit Spielernamen und Elo → F = players.csv z.B. Dr Spectred, 8

```
readPlayers(fileName :: String) → Vector{Player}
```
↑ F

```
writePlayers(fileName :: String, players :: Vector{Players})
```
→ Speichert CSV F

CSV mit neuen(!) Matches → matches.csv → z.B. Dr Spectred, Malf
CSV mit neuer Results, passend zu matches.csv → results.csv
} zum Einlesen
CSV mit allen Results → allResults.csv, evtl. mit Leerzeilen für Matchdays

→ z.B. 4 (wegen 3:0)

Funktionen

readNewMatches
readNewResults → jeweils Vector
readAllResults
writeUpdatedResults → updatet allResults mit results

```
giveExpectedScores(; playerFileName = "players", matchFilename = ...)
givePlayerVal(; playerF... = ..., matchFilename = ..., mode :: String)
```
"Elo" / "Exp"

```
if mode == Elo
    for player in players
        → rating - cost
elseif mode == Exp
    ...
    → expectedScore/cost
```

```
updateElos!(; playerFile, matchFile, resultFile)
updateAll!(; -"- , allResultsFile)
updateResults(; match, results, allResults)
```